Lab 6 Report
Path Finding: Grid-based Path Planning, Sampling-based Path Planning, and Pure Pursuit Controller Integration

Team 12

Alex Chen
Yaqi Li
Daniel Odenthal
Simdi Okoroafor
Tina Zhang

Robotics: Science and Systems

April 25, 2026

1 Introduction

In order to have successful autonomous navigation, the racecar not only needs to accurately and efficiently predict its current location on a pre-established map, but also plan and traverse an unobstructed path towards a goal location, if such a path exists.

In this lab, we designed and implemented a path planning algorithm alongside a pure pursuit controller to autonomously navigate the racecar towards an end goal location, safely, efficiently, and accurately.

Over the past few labs, our team developed various algorithms that aid in the autonomous control and navigation of the racecar. This lab builds upon that, utilizing past algorithms to navigate the robot without hitting obstacles and localizing the racecar based on sensor inputs. This lab utilizes these algorithms alongside the path planning algorithm and the pure pursuit controller to allow our racecar to navigate from one destination to another.

Our first goal in this lab is to develop and implement a path planning algorithm. Given an ending goal location on a map, the racecar must plan an unobstructed

path from its current location to the goal location. The path planning algorithm must plan around any known obstacles such as pillars, and give enough space between the walls to account for the vehicle size. Our team developed both a grid-based and a sampling-based path planning algorithm, and will evaluate the performance of both. The output from this algorithm is an array of poses that point from the racecar's pose to the ending location, and this is referred to as the path.

The second goal is to have the racecar follow the path, and our team developed a pure pursuit controller to achieve this. The controller constantly looks at a path pose that is at least a certain distance away from the racecar, and moves towards that point. When the goal destination has been reached, the racecar stops moving, and throughout this process, a safety controller is also implemented to prevent collisions with the environment and unplanned obstacles such as people walking nearby.

These components combined will allow our racecar to achieve autonomous driving to a desired location.

2 Technical Approach

2.1 Monte Carlo Localization for Pose Estimation

Path planning requires the car to know its pose on a known map. We implemented the Monte Carlo Localization (MCL) algorithm in Lab 5 where we use a set of N weighted particles $\{x_t^{(i)}, w_t^{(i)}\}_{i=1}^N$ to represent a belief distribution over the car's pose $x_t = (x, y, \theta)$. The MCL algorithm performs motion update, sensor weighting, and resampling on each callback and publishes the weighted mean pose estimate to the planner through the TF2 transform `map` $\leftarrow$ `base_link`.
On each `/odom` message, each particle is propagated forward using the twist (v, v_y, ω) integrated over elapsed time Δt with added Gaussian noise to maintain particle diversity and represent uncertainty in odometry:

$$x' = x + \cos(\theta)\Delta x - \sin(\theta)\Delta y + \mathcal{N}(0,\, \sigma_x) \tag{1}$$

$$y' = y + \sin(\theta)\Delta x + \cos(\theta)\Delta y + \mathcal{N}(0,\, \sigma_y) \tag{2}$$

$$\theta' = \theta + \Delta\theta + \mathcal{N}(0,\, \sigma_\theta) \tag{3}$$

We scaled the motion noise parameters $\sigma_i = \sigma_i^{\text{base}} \times s$, where $\sigma_x^{\text{base}} = \sigma_y^{\text{base}} = \sigma_\theta^{\text{base}} = 0.075$ and s is a scalar to be tuned. Through parameter tuning, we found $s = 0.2$, which gives $\sigma_x = \sigma_y = \sigma_\theta = 0.015$, minimizes the cross-track error root mean square error (CTE RMSE) while preserving sufficient particle diversity.

Furthermore, on each `/scan` message, `PyScanSimulator2D` ray-traces M range values $d^{(i)}$ from each particle's hypothesized pose. Each beam's likelihood is a

Gaussian mixture:

$$p(z_k^{(i)} \mid x^{(i)}, m) = \alpha_{\text{hit}}\, p_{\text{hit}} + \alpha_{\text{short}}\, p_{\text{short}} + \alpha_{\max}\, p_{\max} + \alpha_{\text{rand}}\, p_{\text{rand}}, \tag{4}$$

where p_{hit} models correct obstacle detection, p_{short} models spurious short returns, $p_{\max}$ models missed measurements, and p_{rand} models random noise. To avoid evaluating these distributions at runtime, a lookup table of precomputed conditional probability values is built at initialization, allowing $O(1)$ lookups. Particle weights are then computed via log-sum to prevent numerical underflow:

$$w^{(i)} = \exp\left(\sum_{k=1}^{M} \log\, p(z_k^{(i)} \mid x^{(i)}, m)\right). \tag{5}$$

We resample when the effective sample size $N_{\text{eff}} = 1/\sum_i (w^{(i)})^2$ falls below $0.55N$ in order to prevent unnecessary loss of particle diversity. The published pose estimate uses the weighted circular mean to avoid angle-wrapping errors near $\pm\pi$:

$$\hat{\theta} = \operatorname{atan2}\!\left(\textstyle\sum_i w^{(i)} \sin\theta^{(i)},\ \sum_i w^{(i)} \cos\theta^{(i)}\right), \tag{6}$$

with (x, y) as the weighted arithmetic mean. We ended choosing a tuned configuration of $N = 200$ particles, $M = 120$ LiDAR beams, and a motion noise scale of $s = 0.2$ to achieve a CTE RMSE below 0.05 m in simulation while maintaining real-time filter rates.

2.2 Grid-based Path Planning: A* Search

Grid-based path planning discretizes the occupancy map into a uniform grid to use as a graph and uses a graph-search algorithm to find a shortest path from the robot's current position to a goal. We receive the map as an `OccupancyGrid` message and reshape it into a Boolean array $\mathcal{G} \in \{0,1\}^{H \times W}$ where each cell is marked to be occupied if its probability value is greater than 50. Furthermore, we convert world-frame to grid indices using

$$(r,\, c) = \left\lfloor \frac{R_\theta^{-1}\,(\mathbf{p} - \mathbf{o})}{\Delta} \right\rfloor, \tag{7}$$

where $\mathbf{o}$ is the map origin, θ is its yaw, R_θ is the corresponding 2-D rotation matrix, and Δ is the map resolution in meters per cell. When integrated, the robot's start pose is supplied by the MCL filter described in the previous section.

To ensure a margin of safety around obstacles, we apply binary dilation to $\mathcal{G}$ to construct $\mathcal{G}_{\text{inf}}$:

$$\mathcal{G}_{\text{inf}}[r, c] = \max_{|\delta r| \le r_{\text{inf}},\, |\delta c| \le r_{\text{inf}}} \mathcal{G}[r + \delta r,\ c + \delta c], \tag{8}$$

where δr and δc are row and column offsets. A cell is thus marked occupied in $\mathcal{G}_{\text{inf}}$ if any cell within r_{inf} of it is occupied in $\mathcal{G}$. Dilation is computed using

`scipy.ndimage.binary_dilation` in $O(1)$ time. We test inflation radii $r_{\text{inf}} \in \{4, 6, 8, 10\}$ cells and selected $r_{\text{inf}} = 6$, which leads to about 149,000 occupied cells and providing $6 \times 0.05 = 0.30\,\text{m}$ of clearance from every obstacle surface while preserving navigable corridors, as shown in Figure 1. We find that larger radii lead to narrow corridors that make it harder to navigate, and small radii don't sufficiently guarantee complete safety around obstacles. Additionally, we tune the path subsampling step $k \in \{1, 3, 5, 10\}$ and selected $k = 10$, which keeps every 10th waypoint and reduces the published path from 168 points down to 18 while giving the pure-pursuit controller sufficient resolution to track paths accurately.

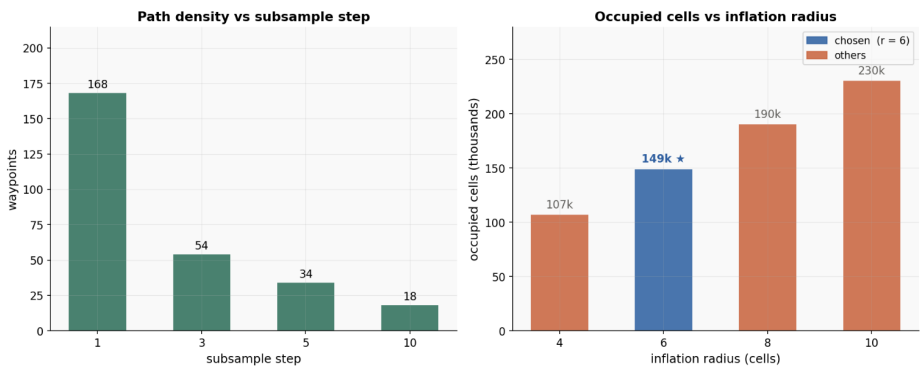

Figure 1: Parameter sweeps for subsampling step and inflation radius. Left is the waypoint count vs. subsampling step k. Increasing k beyond 10 yields diminishing reduction in waypoints. Right is the occupied cell count vs. inflation radius r_{inf}, with the selected value $r = 6$ highlighted. Increasing r beyond 6 rapidly shrinks navigable free space.

We use a grid graph with cells of maximum 8 neighbors where axis-aligned moves cost 1 and diagonal moves cost $\sqrt{2}$, proportional to Euclidean distance. The A* algorithm maintains a min-heap priority queue ordered by

$$f(v) = g(v) + h(v), \tag{9}$$

where $g(v)$ is the actual cost from start to v and $h(v)$ is an admissible heuristic estimate of remaining cost to the goal. Algorithm 1 gives the full procedure.

```
Input: start s, goal g, inflated grid G_inflation, heuristic h
Output: path π or None
Initialize heap with (h(s), s); g-score[s] ← 0;
while heap not empty do
    Pop v with smallest f-score;
    if v = g then
        return reconstruct_path(v)
    end
    for each free 8-neighbor u of v do
        c ← √2 if diagonal else 1;
        g' ← g-score[v] + c;
        if g' < g-score[u] then
            g-score[u] ← g';
            Push (g' + h(u), u) onto heap;
        end
    end
end
return None
```

Algorithm 1: A* on 8-connected inflated grid.

We evaluate four different heuristic functions for the A* algorithm: Manhattan ($|\Delta r| + |\Delta c|$), Euclidean ($\sqrt{(\Delta r)^2 + (\Delta c)^2}$), Octile ($\sqrt{2}\min(|\Delta r|, |\Delta c|) + \big||\Delta r| - |\Delta c|\big|$), and Zero ($h \equiv 0$, equivalent to Dijkstra's algorithm). A heuristic is admissible if it never overestimates the true cost ($h(v) \leq h^*(v)$) and consistent if $h(n) \leq c(n, n') + h(n')$ for every edge. These two properties of the heuristic function guarantee that A* returns an optimal path. We select the Octile heuristic for our A* implementation, as it is admissible and consistent and efficient. We validate this empirically and compare search efficiency across heuristics in Section 3.

2.3 Sampling-based Path Planning: RRT

Sampling-based path planning avoids building a full grid graph and instead explores the space by randomly sampling points. In this lab, we implement the Rapidly-exploring Random Tree (RRT) algorithm, which starts from the robot's current position and grows a tree outward toward randomly chosen points in free space. Compared to grid-based methods like A*, RRT does not always find the optimal path, but it is faster and works well for real-time planning.
The occupancy grid map is converted into a Boolean grid $\mathcal{G} \in \{0, 1\}^{H \times W}$, where cells are marked as occupied if their value is above a threshold. We use the same world-to-grid conversion from Equation 7 and apply obstacle inflation as in Equation 8 to ensure the robot keeps a safe distance from obstacles.

The RRT algorithm works by repeatedly sampling random points in free space. For each sampled point x_{rand}, the algorithm finds the nearest node x_{near} already

in the tree and moves a fixed step size toward it to create a new node x_{new}. With some probability p_{goal}, the algorithm directly samples the goal to help guide the tree in the right direction. Before adding a new node, we check that the straight-line path between x_{near} and x_{new} is collision-free. If it is, the node is added to the tree. This process continues until a node is close enough to the goal and can be connected to it safely. The final path is then reconstructed by following parent pointers from the goal back to the start.
After generating a path, we apply a simple shortcutting step to make it shorter and smoother. We randomly pick pairs of points along the path and check if a straight line between them is collision-free. If it is, we remove the intermediate points. This helps reduce unnecessary turns and improves the overall path quality. Now, taking a closer look at the algorithm itself:

```
Input: start s, goal g, inflated grid G_inf
Output: path π or None
Initialize tree with node s;
for i = 1 to N_max do
    Sample x_rand from free space (with probability p_goal, set x_rand = g);
    Find nearest node x_near in tree;
    Steer toward x_rand to generate x_new;
    if x_new is in free space and edge (x_near, x_new) is collision-free then
        Add x_new to tree with parent x_near;
        if ||x_new − g|| < r_goal and edge to g is collision-free then
            Add g to tree;
            return reconstructed path from s to g;
        end
    end
end
return None
```

Algorithm 2: Rapidly-exploring Random Tree (RRT) for path planning.

Compared to A*, RRT is faster and easier to run in real time, but the paths are usually longer, less optimal and less consistent. We explore this tradeoff further in Section 3.

2.4 Pure Pursuit Integration

We implemented a Pure Pursuit controller to enable the vehicle to track a predefined trajectory using real-time localization data. The controller operates by continuously updating steering commands based on the vehicle's current pose and a dynamically selected target point along the path.

At each timestep, the controller first receives the vehicle's position and orientation from the localization system. This orientation is then converted into a yaw angle, which is used to relate global coordinates to the vehicle's local reference

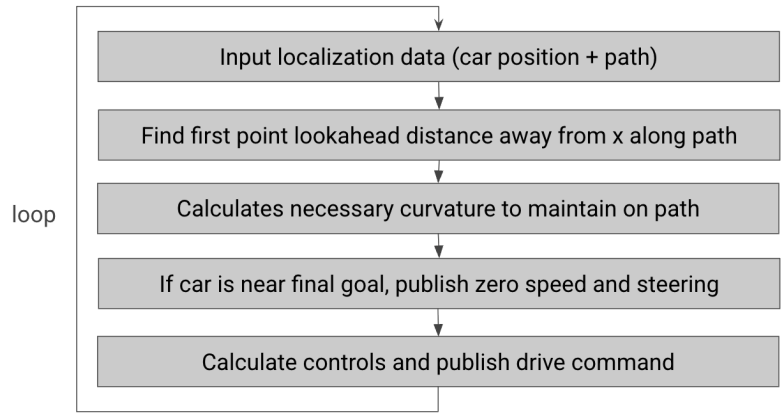

Figure 2: Overview of the Pure Pursuit control algorithm.

frame. The controller checks whether the vehicle has reached the final goal by computing the Euclidean distance to the last waypoint. If the distance is within a specified tolerance, zero speed and zero steering will be published and the car will reach a stop.

To determine the tracking target, the controller identifies the waypoint on the trajectory closest to the current vehicle position. The controller selects the first waypoint that is at least a fixed lookahead distance away from the vehicle. If no such point exists, the final waypoint is used as the goal point. This ensures that the vehicle always tracks a point ahead of its current position.

The selected goal point is then transformed from the global frame into the vehicle's local frame. Using this transformation, the controller computes the curvature required to reach the goal point:

$$\kappa = \frac{2y}{L_d^2} \tag{10}$$

where y is the lateral offset of the goal point in the vehicle frame and L_d is the lookahead distance.

The steering angle is then calculated using the vehicle's wheelbase:

$$\delta = \tan^{-1}(L\kappa) \tag{11}$$

where L is the wheelbase length. A steering limit is applied to prevent excessively large steering commands that could destabilize the vehicle.

Lastly, the controller publishes a drive command consisting of a constant forward velocity and the computed steering angle. This process repeats continuously to

allow the vehicle to smoothly follow the trajectory.

Our implementation of Pure Pursuit provides an effective method for trajectory tracking by converting odometry into real-time control commands.

3 Experimental Evaluation

3.1 Simulation Validation: A* Heuristic Selection and Algorithm Comparison

We run a baseline comparison and record the median number of heap pops as a representation of search effort. As shown in Figure 3, Dijkstra's algorithm expands a median of of 100,200 nodes while A* with Manhattan, Euclidean, and Octile heuristics expand 600, 7700, and 4400 respectively. All three A* variants return the same costs for paths to Dijkstra, which confirms path optimality, as Dijkstra's algorithm is guaranteed to find the optimal path in a graph given non-negative edge weights. We choose the Octile heuristic for our A* algorithm, as it expands far fewer nodes than Dijkstra and Euclidean and is admissible and consistent unlike Manhattan, thus guaranteeing path optimality while minimizing search effort.

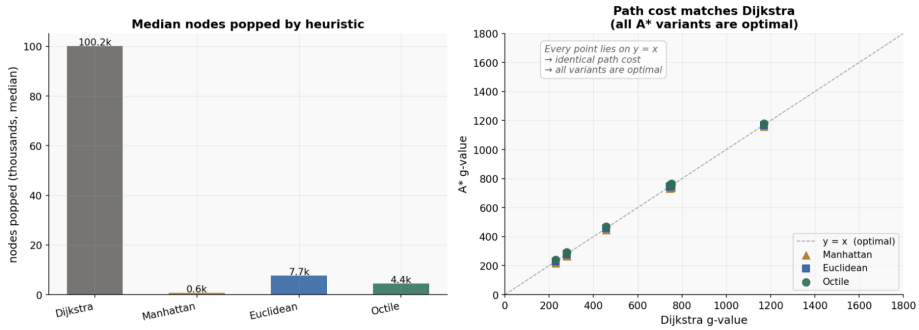

Figure 3: Heuristic comparison across four A* variants. Left is the number of median nodes popped per heuristic. Dijkstra expands 100,200 nodes versus 4400 for Octile. Right is the A* g-value vs. Dijkstra g-value for each heuristic. All points lie on $y = x$, which confirms that every A* variant returns an optimal path identical in cost to Dijkstra.

Before integrating with pure pursuit, we validate whether grid-based or sampling-based path planning is better to use by analyzing their tradeoffs in efficiency and accuracy. Although A* requires a mean planning time of 0.232 seconds compared to 0.072 seconds for RRT, it always outputs guaranteed optimal paths while RRT may output longer paths. In our experiments, we find A* paths with mean length 41.4 m compared to 49.1 m for RRT, as shown in Figure 4.

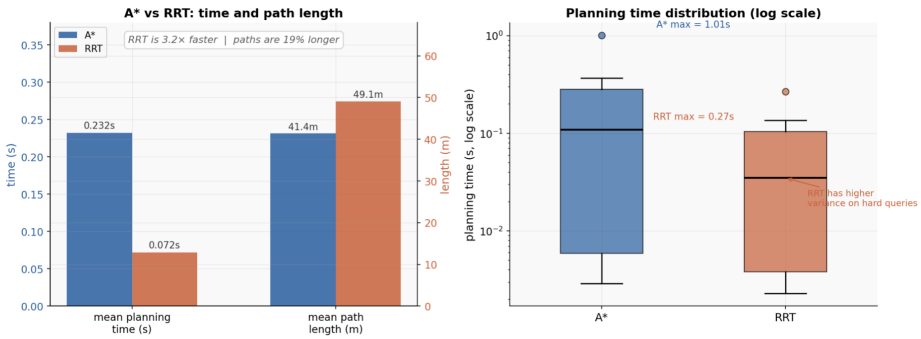

Figure 4: Planning time and path length comparison between A* and RRT. Left is the mean planning time and mean path length for each algorithm. RRT is 3.2 times faster but produces paths 19% longer on average. Right is the planning time distribution on log scale. A* has bounded worst-case time while RRT shows higher variance.

We choose to integrate the A* algorithm with pure pursuit, because despite a longer planning time, path planning is done only once per goal, and the 7.7 m reduction in path length directly reduces the distance the car must travel and the risk of accumulating tracking error over a longer trajectory. Thus, we are willing to trade a longer planning time for path optimality.

3.2 Physical Testing with Full Integration

The final integration included the use of sections from Labs 5 and 6 to successfully plan and follow a trajectory, based on a self-localized starting point and a given end position. Once the car was placed in the Stata basement and fed the map of its environment, it was given a beginning pose estimate and used the Monte Carlo Localization algorithm to update its position based on odometry and LiDAR data. The team was then able to select a desired end position on the map, which triggered the path planner to find a near-optimal trajectory to the position using its localized starting position in the A* algorithm. Finally, the robot used the updated locations with the pure pursuit controller to follow the trajectory until it reached the desired position.

In order to test the full integration of sections on the physical race car, the team compared measurements from distinct landmarks, seen in Figure 5, in the visualization system against measurements in the physical world. The team compared these measurements with ten different samples, as seen in Table 1. The mean absolute position error was found to be 4.2 ± 0.7 cm, meaning the race car was successful in tracking itself in the given environment.

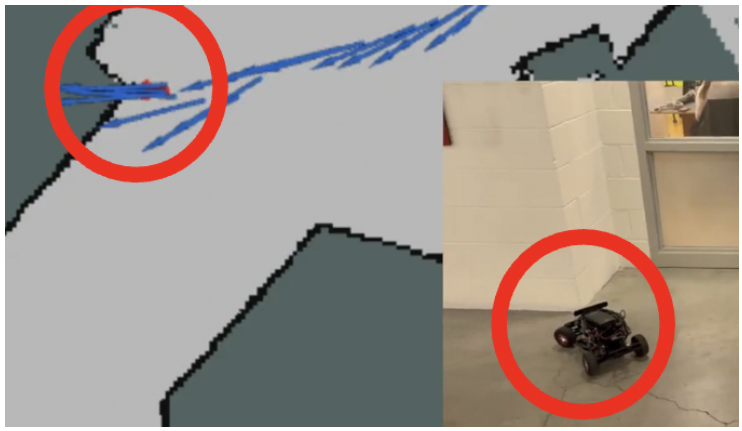

Figure 5: Unique real life landmarks (right), like the corner of a pillar, were matched to the map in simulation (left) to verify successful localization of the race car. The team also collected measurements, using the landmarks as origins, to test the localization accuracy.

Trial	**Sim (m)**	**Real (m)**	**Absolute Error (m)**
1	0.78	0.71	0.07
2	0.82	0.75	0.07
3	0.87	0.81	0.06
4	0.77	0.74	0.03
5	0.81	0.74	0.07
6	1.52	1.49	0.03
7	1.51	1.49	0.02
8	1.49	1.51	0.02
9	1.39	1.41	0.02
10	1.53	1.50	0.03

Table 1: Comparison of onboard map-based position estimates and real-world measurements with corresponding absolute error. The mean absolute position error was 4.2 cm (standard deviation: 2.3 cm).

The team also conducted quantitative testing of the robot's performance, when commanded to follow a trajectory, by observing its behavior along the entirety of the route. The race car stayed within a small limit about the planned trajectory, as seen in Figure 6, and was able to successfully localize and control its motion to achieve the desired position on the map.

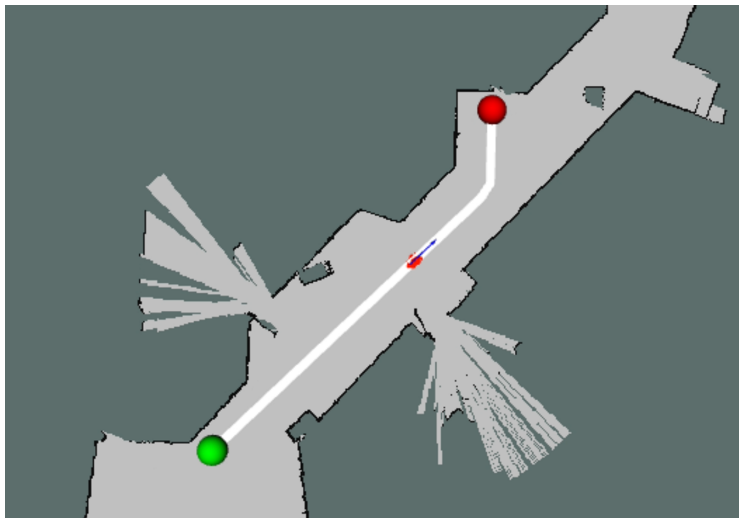

Figure 6: A screenshot of the localized position of the race car (red) closely tracing the planned trajectory (white). The race car is able to accurately localize itself in the environment and use its localized position to continuously adjust its pure pursuit trajectory follower.

4 Conclusion

In this lab, we successfully designed, implemented, and evaluated a grid-based path planning algorithm alongside a pure pursuit controller, in both simulation and in the real world.

By designing two different path planning algorithms, we were able to evaluate the performance of both in terms of runtime and path length. We decided that the it would be worth the extra runtime in exchange for the most optimal path, and chose the A* grid-based algorithm. Through tuning of the pure pursuit controller by varying the lookahead distance of the next point, we were able to achieve smooth navigation for straight paths and were able to turn corners as well. After implementing the algorithms and controller in the real world, we found that the racecar performed well. Compared to previous labs, we were able to gather quantitative data to prove that our implementation works based on measured data.

While the system did perform well, there are still areas for improvement. Future work can focus on improving the algorithm's robustness in tight passages and navigation in less optimal paths that may consist of more jagged segments. In addition, more variety in the testing for the performance of the path planning algorithms as well as more trials in general can lead to even more concrete data analysis and conclusions. These improvements will further strengthen our race-

car's capabilities to navigate the real world in real time.

Looking ahead, our team will be designing algorithms to tackle the final challenge. The final challenge will require us to utilize image processing, localization, autonomous navigation, controller tuning, and many more tools that we have gained throughout this semester. In addition to the algorithms themselves, we will also bring the lessons we have learned throughout the process, such as debugging, testing, data collection and much more, to the table.

5 Lessons Learned

Tina: I learned that it's a lot harder to translate from simulation to the physical world, because there's a lot of factors like noise and physical car lag that we have to take account of. I wish we spent more time tuning the physical settings and doing quantitative analysis on physical testing so we would have more empirical results. Another takeaway is that it's really important to step up and speak up about which module or portion of the lab we should work on and really committing to it. Although as a group we did assign parts, I feel like we could work on meeting the commitments on time or even earlier so it leaves more time for integration and testing.

Simdi: One of my main takeaways from this lab was understanding the tradeoff between different planning approaches. Sampling-based methods like RRT are much faster and easier to run in real time, but they do not guarantee the best path and can sometimes produce inefficient trajectories. This made it clear that choosing the right algorithm depends on the specific goals of the system. I also learned that small implementation details, like collision checking resolution and obstacle inflation, can have a big impact on performance and reliability.

Daniel: I thought this lab was super interesting and it felt really good when we were able to integrate the last lab into this one; being able to select a location on the map in the visualization system and watch the robot figure out how to get there was almost a surreal feeling. I still have a lot to learn on the algorithmic side, but I feel like I am getting pretty good with the physical implementation of the race car. Communication wise, I think the team struggled a little bit coming back from spring break and midterm season, and that resulted in some crunch time at the end of the week. Nonetheless, I think the team has come along well and is ready to tackle the final challenge.

Yaqi: My main takeaways from this lab was that tuning parameters in simulation does not directly translate to good performance in the real world. There are many additional noise and limits in the physical system so it is important to leave sufficient time for testing and tuning on the real vehicle. Another lesson I learned was time management. Our team ended up doing a significant amount of work towards the deadline which made debugging and integration

more stressful and difficult. In the future, I would prioritize scheduling work sessions earlier and reduce last-minute pressure. I also learned to be attentive to group messages and updates, especially when plans or timelines change. Also its important to communicate early if tasks cannot be delivered on time so the team can adjust accordingly.

Alex: One of my main takeaways from this lab is to communicate better and early. Due to a scheduling conflict, I wasn't able to fully complete a module, and I know that impacted the timeline that our team set. We were able to integrate and test, but we didn't have as much time as we would've liked and couldn't test as much depth. Going into the final challenge, meeting deadlines and leaving buffer time is crucial to our team's success. Personally, I will make sure to communicate better with my team. My second main takeaway is that simulation tuning is very different from physical tuning. Tuning the pure pursuit algorithm in simulation yielded great results, but the same tuning did not translate well in the real world.